

CPSC 375 High-Performance Computing

Lecture 12

Thread Safety & POSIX Threads

Thread Safety

- Functions called from a thread must be *thread-safe*
- We identify four (non-disjoint) classes of *thread-unsafe* functions:
 - Class 1: Failing to protect shared variables
 - Class 2: Relying on persistent state across invocations
 - Class 3: Returning pointer to static variable
 - Class 4: Calling thread-unsafe functions

Thread-Unsafe Functions

- Class I: Failing to protect shared variables
 - Fix: Use semaphores
 - Issue: Synchronization operations will slow down the code
 - Example: `goodcnt.c`

Thread-Unsafe Functions (cont)

- Class 2: Relying on persistent state across multiple function invocations
 - Random number generator relies on static state
 - Fix: Rewrite function so that caller passes in all necessary state

```
/* rand - return pseudo-random integer on 0..32767 */
unsigned int next = 1;
int rand(void)
{
    next = next*1103515245 + 12345;
    return (unsigned int)(next/65536) % 32768;
}

/* srand - set seed for rand() */
void srand(unsigned int seed)
{
    next = seed;
}
```

Thread-Unsafe Functions (cont)

- **Class 3: Returning pointer to static variable**

- **Fixes:**

1. Rewrite code so caller passes a pointer to a struct

- Issue: Requires changes in caller and callee

2. *Lock-and-copy*

- Issue: Requires only simple changes in caller (and none in callee)
- However, caller must free memory

```
struct hostent
*gethostbyname(char* name)
{
    static struct hostent h;
    <contact DNS and fill in h>
    return &h;
}
```

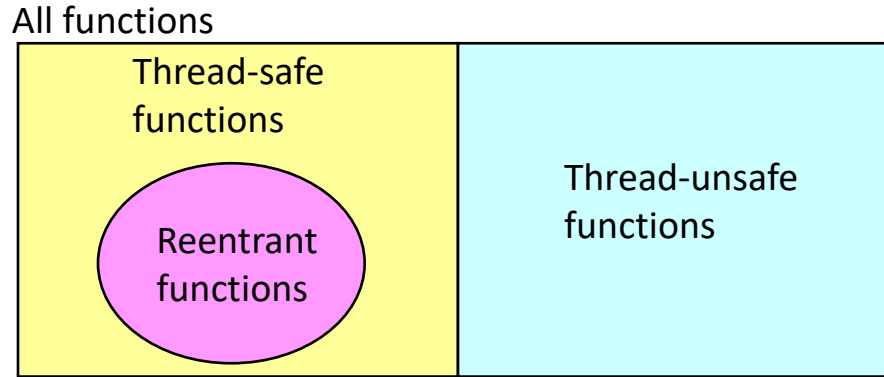
```
hostp = malloc(...);
gethostbyname_r(name, hostp);
```

```
struct hostent
*gethostbyname_ts(char *p)
{
    struct hostent *q = Malloc(...);
    P(&mutex); /* lock */
    p = gethostbyname(name);
    *q = *p; /* copy */
    V(&mutex);
    return q;
}
```

Thread-Unsafe Functions

- Class 4: Calling thread-unsafe functions
 - Calling one thread-unsafe function makes an entire function thread-unsafe
 - Fix: Modify the function so it calls only thread-safe functions

Reentrant Functions



- A function is *reentrant* iff it accesses **NO** shared variables when called from multiple threads
 - The fixes to Class 2 and 3 thread-unsafe functions require modifying the function to make it reentrant
 - Only the first fix for Class 3 is reentrant.

Thread-Safe Library Functions

- Most functions in the Standard C Library are thread-safe
 - Examples: `malloc`, `free`, `printf`, `scanf`
- All Unix system calls are thread-safe
- Library calls that aren't thread-safe:

Thread-unsafe function	Class	Reentrant version
<code>rand</code>	2	<code>rand_r</code>
<code>strtok</code>	2	<code>strtok_r</code>
<code>asctime</code>	3	<code>asctime_r</code>
<code>ctime</code>	3	<code>ctime_r</code>
<code>gethostbyaddr</code>	3	<code>gethostbyaddr_r</code>
<code>gethostbyname</code>	3	<code>gethostbyname_r</code>
<code>inet_ntoa</code>	3	(none)
<code>localtime</code>	3	<code>localtime_r</code>



POSIX Threads

- A standard for Unix-like operating systems known as *Pthreads*
- A library that can be linked with C programs.
- Specifies an application programming interface (API) for multi-threaded programming.

Pthreads Interface

- creating and *reaping* threads
 - `pthread_create()`
 - `pthread_join()`
- determining thread ID
 - `pthread_self()`
- terminating threads
 - `pthread_cancel()`
 - `pthread_exit()`
 - `exit()` [terminates all threads]
- *synchronizing* access to shared variables
 - `pthread_mutex_init`
 - `pthread_mutex_[un]lock`
 - `pthread_cond_init`
 - `pthread_cond_[timed]wait`

Pointers to Functions in C

- C doesn't require that pointers point only to *data*; it's also possible to have pointers to *functions*.
- Functions occupy memory locations, so every function has an address.
- We can use function pointers in much the same way we use pointers to data.
- Passing a function pointer as an argument is fairly common.

Function Pointers as Arguments

- A function named `integrate` that integrates a mathematical function `f` can be made as general as possible by passing `f` as an argument.

```
double integrate(double (*f)(double, double),  
                 double a, double b);
```

Here, the parentheses around `*f` indicate that `f` is a pointer to a function.

Function Pointers as Arguments

- A call to `integrate` that integrates the `sin` (sine) function from 0 to $\pi/2$:

```
result = integrate(sin, 0.0, PI/2);
```

- Within the body of `integrate`, we can call the function that `f` points to:

```
y = (*f) (a, b);
```

Starting the Threads

For each thread, call

```
int pthread_create(  
    pthread_t *thread,    /* thread handler */  
    pthread_attr_t *attr, /* thread attributes */  
    void * (*start_routine) (void *), /* thread function */  
    void *arg); /* arguments for thread function */
```

Starting the Threads

```
int pthread_create(  
    pthread_t *thread,  
    pthread_attr_t *attr,  
    void * (*start_routine) (void *),  
    void * arg);
```

Allocate before calling.

We won't be using it,
so we just pass NULL.

↑
Pointer to the argument
that should be passed to
the function *start_routine*.

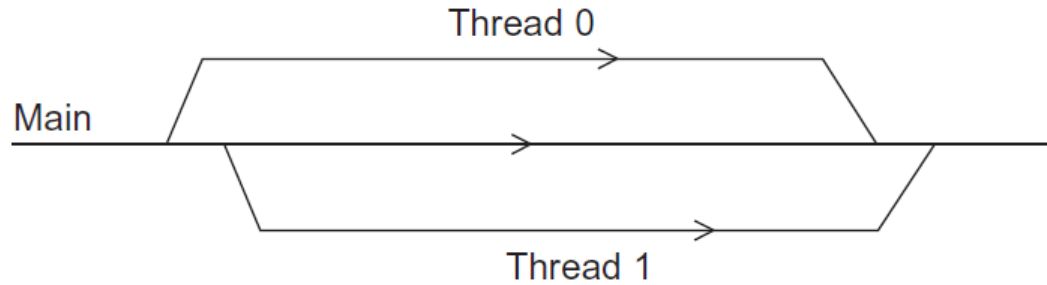
The function (task)
that the thread is to
run.

Function started by `pthread_create`

```
void *start_routine (void *arg) ;
```

- `void *` can be cast to any pointer type in C.
- *arg* can point to a list containing one or more values needed by *start_routine*.

Running Threads



Main thread forks and *joins* two threads.

Stopping Threads

- We call the function `pthread_join` once for each thread.
- A single call to `pthread_join` will wait for the thread associated with the `pthread_t` object to complete.

Demo I

- Log in to a lab machine.
- **Copy** `pth_hello.c`:
\$ `cp /home/pyoon/cpsc375/pth_hello.c .`
- **Compile**:
\$ `gcc -o pth_hello pth_hello.c -lpthread`
- **Run to print “hello” using 10 threads**:
\$ `./pth_hello 10`

Demo II

- **Copy** `pth_add.c`:
`$ cp /home/pyoon/cpsc375/pth_add.c .`
- **Compile**:
`$ gcc -o pth_add pth_add.c -lpthread`
- **Run to add 1~1,000,000 using 8 threads**:
`$ ./pth_add 1000000 8`
- **Compare running times with various numbers of threads**:
`$ time ./pth_add 1000000 10`

